COMPSYS 303 Assignment

Designing a Traffic Light Controller in SCCharts:
Timed SCCharts versus an External Multiform-Time Reference

Part 1: Due Week 7 — Part 2: Due Week 12

Exact due dates are available on Canvas – please check there.

Learning Outcomes

On completing this assignment, students will be able to:

- Understand the synchronous approach to concurrency and apply it to a reactive control problem.
- Design a reactive system directly in SCCharts, translating an informal behavioural specification into states, regions, transitions, and their guards, without being given a reference state machine.
- Understand how physical time can be handled in a synchronous language in two distinct ways: as a first-class quantity using the clocks of Timed SCCharts, and by the traditional multiform-time approach of counting occurrences of an external timing reference.
- Design, implement, and simulate two functionally-equivalent controllers built on these two different notions of time, and reason about the trade-offs between them (timing accuracy, jitter, tick count / computational effort, and modelling clarity).
- Understand the design of a safety-relevant embedded controller in which incorrect timing behaviour has real consequences.

Overview

For this assignment you are required to design and implement a traffic light controller for the intersection of two roads, built up over four tasks of increasing capability. Unlike previous versions of this assignment, you are not given a state machine to implement. You are given only a description of the required behaviour; designing the corresponding SCChart – the states, the concurrent regions, and the reactive interface – is your task, and its quality is assessed.

The central theme of this assignment is how time is represented in a synchronous program. You will implement two versions of the controller that behave identically but handle time differently, and compare them:

1. **Version A – Timed SCCharts.** All time-outs ($t_1, \ldots, t_6$, the pedestrian and camera timings) are expressed using Timed SCCharts clocks, with elapsed time compared against the time-out value directly in the transition guards. This version is designed and tested in simulation using the online SCCharts simulation tool provided for this course.

2. **Version B – External timer (multiform time).** The SCChart is given no clocks. Instead, time-outs are driven by an external timer, using the Nios II timer-interrupt facility exactly as in the original version of this assignment: the SCChart measures each time-out by counting occurrences of the timer signal delivered from outside the chart. This is the traditional multiform notion of time – time is just another event the program counts – and requires a reactive interface between the generated SCCharts code and the Nios II timer (see Section 1.3 and Part 2 of the submission).

A note on what is, and is not, provided. You are given the behaviour of each task (which lights are on in each state, the ordering of states, what each input does, and which quantities are timed). You are not given the SCChart, the set of states, or the region structure – designing these is the assignment. Time-out values and output encodings are specified; every design decision (how many states, how regions communicate, where each delayed transition goes, how the external timer is counted) is yours to make and to justify in your report.

1 Background

1.1 The Intersection

Consider a traffic light controller for the intersection of two roads: one running north–south (NS) and one running east–west (EW). Each direction has a traffic light with three lamps: red (R), green (G) and yellow (Y). Throughout this document, state and output notation gives the NS light first and the EW light second, so that G-R means NS is green and EW is red. The controller must never allow both directions to show green (or green/yellow) at the same time; the only safe state in which the controller may switch between operating modes is the all-red state R-R.

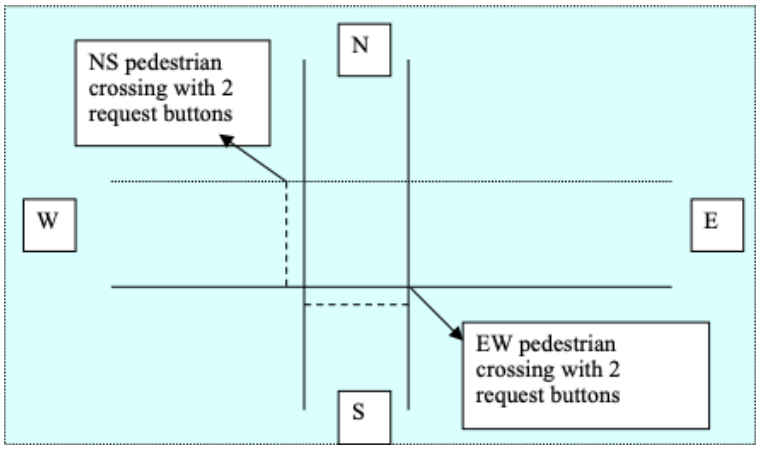


Figure 1: The plant: the four-way intersection of the NS and EW roads, showing the traffic lights, the pedestrian crossings, and the approach lanes controlled by the SCChart.

The six lamps are mapped to the six low bits of the red LED output as shown in Table 1. The two remaining LEDs are used later for the pedestrian lights.

Light	EW-R	EW-Y	EW-G	NS-R	NS-Y	NS-G
LED bit	5	4	3	2	1	0

Table 1: Mapping of the six traffic lamps to red-LED bits.

1.2 Time in a Synchronous Program

An SCChart executes as a sequence of discrete, synchronous ticks. Conceptually, each tick takes zero time – an abstraction that gives the synchronous model its deterministic, race-free semantics. The question this assignment explores is how a program that lives in this tick-by-tick world should measure real, physical time. Two fundamentally different answers are compared below.

Multiform time (Version B): counting an external signal

The traditional synchronous answer is that time is *not special*: it is represented as a signal, just like any other physical or digital input.

- **Mechanism.** An external timer – such as the Nios II timer-interrupt facility – raises a periodic timing event at a fixed rate. A time-out duration (for example, 2 s) is expressed by counting occurrences of this signal in a software variable ($count := count + 1$).
- **Generality.** This is the *multiform* notion of time: the same mechanism could equally count metres travelled, litres poured, or milliseconds elapsed – time is simply whichever input event you choose to count.
- **Advantages and limitations.** Multiform time needs no special language syntax or compiler support. Its accuracy, however, is bounded by the rate and phase alignment of the external timer signal: because state transitions do not in general align exactly with timer events, this approach is subject to a quantisation error of up to one full tick period.
- **Implementation and memory footprint.** Because the timer lives outside the SCChart, this version needs an explicit reactive interface – C or assembly glue code – binding the Nios II timer interrupt to a chart input. In terms of memory, every concurrently timed state requires its own counter variable, which must be reset ($count := 0$) on entry to that state.

First-class time (Version A): Timed SCCharts clocks

Timed SCCharts extend the language with *clock variables*, drawing on the formal semantics of timed automata.

- **Mechanism.** Time is treated as a continuous physical quantity rather than a discrete event to be counted. On entry to a state, a clock is reset: conceptually this records an entry timestamp t_{entry} against the underlying physical clock, rather than resetting or reprogramming any hardware register. A transition guard then compares elapsed time directly against the required time-out, e.g. $t_{\text{current}} - t_{\text{entry}} \geq t_{\text{target}}$.
- **Advantages.** Clocks give higher timing precision – bounded only by the resolution of the underlying processor clock rather than by a periodic interrupt – and produce transition guards that read directly as time-outs. Combined with dynamic ticks, Timed SCCharts also avoid the "busy" recomputation that a counted-signal approach incurs when it must react to every occurrence of a high-frequency timer.
- **Implementation and memory footprint.** The SCChart compiler and runtime handle the underlying timer driver and glue code directly. Rather than a hand-managed integer counter per timed state, each clock is represented by a single stored entry timestamp; this scales cleanly across concurrent regions without those regions competing for shared counting hardware.

A worked, hard-real-time demonstration of Timed SCCharts, continuous clocks, and dynamic ticks – built around a pair of stepper motors that must remain synchronised to microsecond accuracy – is provided as accompanying reading [Boysen et al.]. Pay close attention to how its clock variables and time-out guards (for example the `BLIND_TIME` and `OFF_TIME` constraints) are structured, as a model for the style expected in Version A.

1.3 SCCharts Constructs You Will Need

Recall from lecture that a transition marked *immediate* fires in the same tick as its source state is entered (combinational), whereas a transition with no *immediate* keyword is *delayed* and fires from the next tick (a clocked register). Every closed loop in a controller – including each cycle of the traffic light – must contain exactly one delayed transition; making every transition around a cycle immediate creates a causality error (an algebraic loop) that the compiler will reject.

Concurrent behaviour is expressed with *regions*, each an independently-evolving state machine advancing on the same shared tick. You will need regions from the pedestrian task onward. The reactive skeleton used throughout is:

$$\text{SensorStage} \xrightarrow{\text{immediate}} \text{ControllerStage} \xrightarrow{\text{immediate}} \text{PlantStage} \xrightarrow{\text{delayed}} \text{SensorStage}$$

2 Your Task

You will build the controller in four tasks of increasing capability, exactly as in the classic version of this assignment, but designing every SCChart yourself. Each task must be implemented twice – once as Version A (Timed SCCharts, clocks, in the simulator) and once as Version B (external Nios II timer, counting the timer-interrupt events). You may keep the two versions as separate charts, or use a single chart with a switch that selects the timing mechanism, provided both behaviours are demonstrable. Design the light-switching logic once and reuse it; only the timing mechanism should differ between the two versions.

Mode switching. All four tasks are hosted in one system, and the user must be able to switch between them at any time (in simulation, via an input selecting the active task). A mode change must only take effect when the controller is in the safe R-R state. Designing this safe-switching mechanism is part of the task.

2.1 Task 1: Simple Traffic Light Controller

Design an SCChart that cycles the intersection through the usual sequence of states using six time-outs $t_1, \ldots, t_6$. The controller alternates giving right of way to NS and to EW, with a yellow phase before each change and an all-red state between phases. The behaviour you must realise is:

- NS gets right of way: the light shows G-R for the duration t_1, then Y-R for the duration t_2.
- An all-red safety phase R-R is held for the duration t_3.
- EW gets right of way: the light shows R-G for the duration t_4, then R-Y for the duration t_5.
- An all-red safety phase R-R is held for the duration t_6, after which the cycle repeats.

You must decide the states, the outputs each state drives onto the LED bits of Table 1, and where the single delayed transition around the cycle belongs. For Version A, each time-out is a comparison of a clock against the corresponding t value. For Version B, each time-out is reached

by counting the required number of external timing events. Make the six time-out values named constants so they are easy to change (see Section 3).

Suggested checks: confirm the cycle never shows two non-red directions at once; confirm each phase lasts the intended number of ticks / time units; and, for Version B, confirm that changing the external timer's rate rescales the durations as you would expect.

2.2 Task 2: Pedestrian Traffic Light Controller

Extend Task 1 with pedestrian lights and pedestrian request buttons. The natural design (which you must produce yourself) uses three concurrent regions: one region that runs the intersection lights and the two pedestrian lights, and two further regions that each latch a pedestrian request coming from an external input, modelled as the events NSraised and EWraised. The request-handling regions communicate with the light-controlling region through shared variables.

- The two remaining LEDs represent the NS and EW pedestrian lights.
- The two pedestrian request inputs are modelled as the external events NSraised and EWraised (in simulation, two boolean inputs).
- A raised pedestrian request must be remembered until it has been served by the corresponding direction's green phase, then cleared.

This task should reuse the light-switching logic of Task 1 with minimal change – that reuse is the point of the concurrent-region structure. Because two regions and the main loop may touch the same shared request variables, you must be explicit in your design about how simultaneous access is resolved; a clean approach is to only read a shared value on the tick after it is written, avoiding any dependence on within-tick write/read ordering. The timing of the pedestrian walk phase must, like every other time-out, be implemented via a clock in Version A and via the counted external signal in Version B.

2.3 Task 3: Configurable Traffic Light Controller

In Tasks 1 and 2 the time-outs are fixed. A real controller can be reconfigured – for example at night, when traffic is light. In this task the six time-out values are supplied at run time and loaded only when the controller is safely in R-R.

In the original hardware assignment the new values arrived over UART. In simulation, model the configuration input as a string / sequence delivered to the chart, using the same packet format:

`#,#,#,#,#,#[\r]\n`

- `#` – a 1–4 digit integer time-out value
- `,` – separates the six values
- `\r` – ignored (may or may not be present)
- `\n` – end-of-packet: signals the six values are complete

On receiving a complete, valid packet of six values, the controller transfers them into the six time-out constants and resumes normal operation from the R-R state. If six valid numbers are not received, it keeps waiting. Consider carefully how this parsing interacts with the two timing mechanisms: reconfiguration changes the t values that Version A compares clocks against and that Version B counts up to. Discuss in your report whether reconfiguration is any harder to get right under one timing model than the other.

2.4 Task 4: Red-Light Camera

Augment the controller with a red-light camera that watches for a vehicle still in the intersection during a light change. The camera is armed when a direction is yellow; if a vehicle is still present after a predefined time, a snapshot is taken.

- A vehicle entering and leaving the intersection is simulated by successive presses of a single input: odd presses = vehicle enters, even presses = vehicle leaves. Assume at most one vehicle is in the intersection at any time.
- When the camera timer is armed and starts, print "Camera activated".
- If the vehicle is still present when the camera time-out expires, print "Snapshot taken".
- When the vehicle leaves, print "Vehicle left".

The camera introduces another independent time-out – the dwell time a vehicle is allowed before a snapshot is taken – which again must be realised as a clock in Version A and as a counted external signal in Version B. This task is where the two approaches are most interesting to compare, because the camera time-out now runs concurrently with the light time-outs: in Version A you have two independent clocks; in Version B you must count the same (or a second) external timing signal for two purposes at once. Reflect on which is simpler and why.

3 Configurability Requirement

As with other assignments in this course, timing values must not be buried inside the control logic. Declare the six time-outs $t_1, \ldots, t_6$, the pedestrian walk time, and the camera dwell time as named constants at the top of your chart(s), so that they can be changed – and, in Task 3, overwritten at run time – without hunting through the design. For Version B, the rate of the external timing signal should likewise be a single named quantity, since the meaning of every count depends on it.

4 Required Comparison: Timed vs. Multiform-Time

The comparison of Version A and Version B is a required, assessed part of this assignment, not an afterthought. For at least Tasks 1 and 4 (the simplest and the most concurrent), run both versions under matching conditions – Version A in the simulator, Version B on the Nios II board – and report on the following:

1. **Correctness and equivalence.** Do both versions produce identical light sequences? Where they differ, by how much, and why?
2. **Timing accuracy and jitter.** How closely does each version hit the intended durations? For Version B, how does accuracy depend on the rate of the external timing signal, and on how the count aligns with the ticks?
3. **Computational effort.** Contrast how often each version must compute a tick / how much work is "wasted". Relate this to the distinction between counting a frequent timing signal and reacting only when a clock actually expires.
4. **Modelling clarity.** Which version was easier to design, read, and get correct – particularly once multiple independent time-outs run concurrently (Task 4)? Where did each approach create or remove complexity?

5. **Reconfiguration (Task 3).** Did run-time reconfiguration of the time-outs interact differently with the two timing mechanisms?

Support your comparison with concrete evidence from your simulations (traces, tick counts, measured versus intended durations). A good report will connect what you observe back to the underlying reason: that Version A treats time as a first-class quantity while Version B reconstructs it by counting an external event.

5 Report

Write a design report that lets a reader understand and reconstruct your work. It may be single- or double-column, 12-point font, and should be around five pages. It should cover:

- Your SCChart designs: how you decomposed each task into states and regions, and the reasoning behind the structure (including where each delayed transition sits and how the concurrent regions communicate).
- How each of the two timing mechanisms was realised, with reference to your charts.
- The Version A versus Version B comparison of Section 4, backed by simulation evidence.
- Any difficulties encountered translating the behavioural specification into SCCharts, and any avenues for future work.
- A breakdown of how long (in hours) each part of the project took (design, each task, the two timing versions, the comparison, documentation).

Workload table. Include a short table of hours spent and how the workload was shared, to be presented at your demonstration:

	Member 1	Member 2
Time spent	hrs	hrs
Design / logic	%	%
Implementation	%	%
Debugging & simulation	%	%

6 Submission

The assignment is submitted in two parts through Canvas.

6.1 Part 1 (Due Week 7)

Your Timed SCCharts (Version A) for Task 1 and Task 2, tested against the online simulator. Submit your `.sctx` file(s) through Canvas in Week 7. (The external-timer Version B and the Nios II board deliverables are submitted in Part 2.)

6.2 Part 2 (Due Week 12)

Part 2 has three components, all in Week 12: a demonstration and interview, your code, and your report.

Demonstration and interview. You will demonstrate your working design in Week 12 and be interviewed on it. The interview covers the design decisions behind your SCCharts and your

Version A versus Version B comparison, so both group members should be able to explain any part of the submitted work.

Code. The completed assignment, submitted as a single zip file named by your group number (e.g. `group_1.zip`), due in Week 12, containing:

- Your SCChart (`.sctx`) file(s) for all four tasks, covering both Version A (Timed SCCharts) and Version B (external Nios II timer).
- The Nios II project folder and the BSP project files for your Version B implementation.
- The `.pof` and `.sopcinfo` files used with your design.
- Any documentation (e.g. a README describing which switch/input selects which task and which selects the timing version).
- Your evidence for the Section 4 comparison (simulator traces for Version A, and measured results from the board for Version B), if not already contained in the report.

Report. Your report is submitted separately through Canvas as a PDF, also due in Week 12.

Check Canvas for further information and the exact dates and times of the demonstration, interview, and all submission deadlines.

Good luck!

References

Andreas Boysen, Alexander Schulz-Rosengarten, and Reinhard von Hanxleden. A hard real time demo for dynamic ticks and timed SCCharts. Technical report, Kiel University, Kiel, Germany. Accompanying reading for this assignment.